Comportamiento del servidor

#	Hallazgo (prueba)	Consecuencia
1	Los JSON usan camelCase (primerApellido, curpCliente, clavePoliza), no snake_case (A2, B1).	Mis @JsonProperty("primer_apellido") estaban mal y habrían dejado los campos en null.
2	Las fechas de cliente y beneficiario vienen como timestamp: 1990-01-01T06:00:00.000+00:00 (A2, D7).	No se pueden leer como LocalDate. Hay que convertirlas. En verano de 2003 llegó T05:00, así que el servidor guarda medianoche de la Ciudad de México.
3	"No existe" es 200 con cuerpo vacío, nunca 404 (A1, B3, D3). Las listas vacías son [] (C3, C4, C10).	Cuerpo vacío es igual a Optional.empty(). Un 404 real sería ruta equivocada.
4	POST es upsert: repetir la CURP, la clave de póliza o el beneficiario sobrescribe en silencio (A4, B4, D9).	Antes de crear, el caso de uso debe verificar que no exista.
5	Un rechazo del servidor es 200 con cuerpo vacío: póliza con cliente inexistente (B6) o fecha inválida (A11).	En escrituras, cuerpo vacío es igual a rechazo.
6	El servidor casi no valida: acepta CURP de 9 caracteres (A10, y dejó el cliente basura T404CORTA), tipo 7 (B8), monto negativo (B10), monto con 3 decimales (B9), porcentajes 0 y 101 (D7, D8) y suma de 140% (D6).	Todas las validaciones son nuestras, en el dominio.

7	El POST devuelve lo que enviaste, no lo guardado: respondió 100.555 y el listado muestra 100.56 (B9 contra C1).	La relectura con GET después de escribir es necesaria. Es evidencia concreta para el reporte.
8	PUT /cliente falla solo sin segundo apellido.	El POST sin segundo apellido sí lo borra.
9	PUT /poliza sí funciona (B16). Si la póliza no existe responde 200 vacío y no la crea (B18, B19).	Se verifica la existencia antes.
10	Hay 500 por datos inválidos: monto de más de 9 dígitos (B11) y beneficiario sobre póliza inexistente (D11).	Mi código mapeaba todo 5xx a "no disponible" (503), y estaba mal. Solo 502, 503, 504 y fallos de red son "no disponible".
11	DELETE devuelve 200 aunque no exista (E4) y borra en cascada a los beneficiarios (E1 a E3).	Verificar antes que existe y, después, que ya no está.
12	Una / en un dato (%2F) produce 400 en HTML de Tomcat (B12). Acentos y ñ funcionan (A6, B13).	Prohibir / y \ en todo texto que va en la ruta.
13	Omitir el segundo apellido funciona. En clientes llega null y en beneficiarios llega "". Enviar la palabra null guarda el texto "null" (A8).	Nunca enviar null en la ruta y normalizar "" a null.
14	El beneficiario viene anidado: {"id":{...},"porcentaje":60} (D1).	El DTO necesita un id interno.
15	Hay datos sembrados con CURP en minúsculas (broe550101hdfabc00) y descripciones como	Mi regex [A-Z0-9] los rechazaría. Hay que

	[CONFIDENCIAL].	aceptar minúsculas.
16	Una póliza válida puede convivir con filas basura de otros equipos (como nuestro tipo 7).	Al listar, omitir y registrar en el log las filas inválidas. Un solo registro malo no debe romper toda la consulta.
17	No hay endpoint para listar los beneficiarios de una póliza, ni GET /polizas los incluye (D12).	Es imposible validar que la suma de porcentajes no pase de 100. Se documenta como limitación.
18	Tiempos: normalmente de 0.16 a 0.2 s, con picos de 1 a 2.07 s.	El timeout de lectura de 5 s sirve. Sube a 8 s si quieres margen.
19	GET /cliente/{curp} y GET /polizas/{curp}	distinguen mayúsculas.
20	PUT /poliza con titular inexistente responde 500 y no modifica nada	El POST equivalente responde 200 vacío. Son dos formas de rechazo y el adaptador maneja ambas.
21	los caracteres % y ? en la descripción viajan bien si se codifican.	
22	las búsquedas por nombre son exactas, sensibles a mayúsculas, y el segundo apellido forma parte de la coincidencia	Esto se refleja en la interfaz: pedir el segundo apellido cuando se conozca.

23	POST /poliza con monto 9999999.99 da 500. El límite de monto no es el supuesto	Es evidencia concreta de que las validaciones de capacidad hay que probarlas y no deducirlas.
24	GET /beneficiario y GET /polizas/b/...	exigen el segundo apellido exacto.